ASP.NET Core Cheat Sheet — .NET 10 (LTS) · C# 14 · minimal hosting model

Every concept has a full page in the ASP.NET Reference. Reference verified against: https://learn.microsoft.com/en-us/aspnet/core/

DOTNET CLI

```
dotnet new webapi -o Api      # Minimal API
dotnet new mvc | webapp | blazor
dotnet run  |  dotnet watch
dotnet build -c Release
dotnet publish -c Release -o out
dotnet test
dotnet add package <id>
dotnet ef migrations add Init
dotnet ef database update
```

PROGRAM.CS (MINIMAL HOST)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOpenApi();
builder.Services.AddScoped<ISvc, Svc>();

var app = builder.Build();
if (app.Environment.IsDevelopment())
    app.MapOpenApi();
app.UseHttpsRedirection();
app.MapGet("/hi/{name}", (string name) => $"hi
{name}");
app.Run();
```

No `Startup` class. Top-level statements.

MIDDLEWARE ORDER

```
app.UseExceptionHandler("/error");
app.UseHsts();
app.UseHttpsRedirection();
app.UseStaticFiles();          // or MapStaticAssets()
app.UseRouting();
app.UseCors();
app.UseRateLimiter();
app.UseAuthentication();
app.UseAuthorization();
app.UseOutputCache();
app.MapControllers();
```

CUSTOM MIDDLEWARE

```
public sealed class ReqId(RequestDelegate next)
{
  public async Task InvokeAsync(HttpContext c,
      ILogger<ReqId> log)
  {
    c.Response.Headers["X-Id"] = c.TraceIdentifier;
    await next(c);
  }
}
app.UseMiddleware<ReqId>();
```

Factory style: implement `IMiddleware` + register in DI.

DEPENDENCY INJECTION

```
services.AddSingleton<IClock, SysClock>();  // app
services.AddScoped<IRepo, SqlRepo>();       // per
request
services.AddTransient<IStamp, Stamp>();     // per
resolve
services.AddKeyedScoped<INotifier, Sms>("sms");

class H([FromKeyedServices("sms")] INotifier n);
```

Singleton must not capture a scoped service (validated in Dev).

CONFIGURATION + OPTIONS

```
// precedence: appsettings -> appsettings.{Env}
// -> user secrets -> env vars -> args
builder.Services.AddOptions<SmtpOptions>()
  .Bind(config.GetSection("Smtp"))
  .ValidateDataAnnotations()
  .ValidateOnStart();

class Mailer(IOptions<SmtpOptions> o) { }
// IOptionsSnapshot: per request  IOptionsMonitor:
singleton+OnChange
```

MINIMAL API CRUD

```
var g = app.MapGroup("/todos").WithTags("Todos");
g.MapGet("/", (TodoDb db) =>
TypedResults.Ok(db.All()));
g.MapGet("/{id:int}",
  Results<Ok<Todo>, NotFound> (int id, TodoDb db) =>
    db.Find(id) is {} t ? TypedResults.Ok(t)
                        : TypedResults.NotFound());
g.MapPost("/", (CreateTodo dto, TodoDb db) =>
    TypedResults.Created($"/todos/1",
db.Add(dto.Title)))
 .AddEndpointFilter<ValidationFilter>();
```

CONTROLLER WEB API

```
[ApiController]
[Route("api/[controller]")]
public sealed class OrdersController(IOrders o)
    : ControllerBase
{
  [HttpGet("{id:int}")]
  public async Task<ActionResult<OrderDto>> Get(int
id)
    => await o.FindAsync(id) is {} x ? Ok(x.ToDto())
                                     : NotFound();
}
// AddControllers().AddProblemDetails() -> RFC 9457
```

ROUTING TEMPLATES

```
"/p/{id:int}"      "/p/{id?}"     "/f/{*rest}"
constraints: int guid bool datetime alpha
  length(n) min(n) max(n) range(a,b) regex(..)
app.MapGet("/p/{id:int}", (int id) => id);
var v = app.MapGroup("/v1").RequireAuthorization();
app.MapGet("/robots.txt", () => "").ShortCircuit();
```

RAZOR PAGE

```
@page "{id:int?}"
@model EditModel

public class EditModel(IProducts p) : PageModel
{
  [BindProperty] public ProductInput Input { get; set;
} = new();
  public async Task<IActionResult> OnGetAsync(int? id)
{
    if (id is int i) Input = await p.GetAsync(i);
    return Page();
  }
  public async Task<IActionResult> OnPostAsync() {
    if (!ModelState.IsValid) return Page();
    await p.SaveAsync(Input);
    return RedirectToPage("Index");
  }
}
```

TAG HELPERS

```
<a asp-controller="P" asp-action="Details"
   asp-route-id="@p.Id">@p.Name</a>
<form method="post">
  <input asp-for="Input.Name" />
  <span asp-validation-for="Input.Name"></span>
</form>
<img src="~/logo.png" asp-append-version="true" />
<environment include="Development">...</environment>
```

BLAZOR COMPONENT

```
@page "/counter"
@rendermode InteractiveServer

<button @onclick="Inc">Count: @n</button>
<EditForm Model="Input" OnValidSubmit="Save">
  <DataAnnotationsValidator />
  <InputText @bind-Value="Input.Name" />
</EditForm>

@code {
  int n; void Inc() => n++;
  [Parameter, EditorRequired] public int Id { get;
set; }
}
// modes: Static SSR / Interactive Server / WASM /
Auto
```

SIGNALR

```
public class ChatHub : Hub<IChatClient> {
  public Task Send(string u, string t) =>
    Clients.All.ReceiveMessage(u, t);
}
app.MapHub<ChatHub>("/hubs/chat");

const c = new signalR.HubConnectionBuilder()

.withUrl("/hubs/chat").withAutomaticReconnect().build(
);
c.on("ReceiveMessage", render);
await c.start();
await c.invoke("Send", "alice", "hi");
```

EF CORE

```
builder.Services.AddDbContext<AppDb>(o =>

o.UseSqlServer(config.GetConnectionString("Default")))
;

var items = await db.Products.AsNoTracking()
  .Where(p => p.Price < 100)
  .Include(p => p.Tags)
  .OrderBy(p => p.Name).ToListAsync(ct);

db.Products.Add(new(){ Name = "W" });
await db.SaveChangesAsync(ct);   // unit of work
// dotnet ef migrations add X ; dotnet ef database
update
```

AUTH & AUTHORIZATION

```
builder.Services.AddAuthentication("Cookies")
  .AddCookie().AddJwtBearer();

builder.Services.AddAuthorizationBuilder()
  .AddPolicy("Adult", p => p.RequireClaim("age"))
  .AddPolicy("Vet", p => p.AddRequirements(
      new MinTenure(5)));

app.UseAuthentication();
app.UseAuthorization();
app.MapGet("/x", H).RequireAuthorization("Adult");
// [Authorize(Policy="Adult")]  [AllowAnonymous]
```

ERRORS + LOGGING

```
app.UseExceptionHandler("/error");
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<NotFoundHandler>
();

logger.LogInformation(
  "Placed {OrderId} for {Total:C}", id, total);
using (logger.BeginScope("Req:{Id}",
ctx.TraceIdentifier))
{ /* structured message templates */ }
```

CACHING + RATE LIMITING

```
builder.Services.AddOutputCache(o => o.AddBasePolicy(
  b => b.Expire(TimeSpan.FromSeconds(30))));
app.UseOutputCache();
app.MapGet("/cat", GetCat).CacheOutput();

builder.Services.AddRateLimiter(o =>
o.AddFixedWindowLimiter(
  "api", x => { x.Window = TimeSpan.FromSeconds(10);
               x.PermitLimit = 100; }));
app.UseRateLimiter();
// HybridCache: AddHybridCache() (L1+L2, stampede-
safe)
```

INTEGRATION TEST

```
public class ApiTests(WebApplicationFactory<Program>
f)
  : IClassFixture<WebApplicationFactory<Program>>
{
  [Fact]
  public async Task Get_ok() {
    var client = f.WithWebHostBuilder(b =>
      b.ConfigureTestServices(s => {
        s.RemoveAll<IPay>();
        s.AddSingleton<IPay, FakePay>();
      })).CreateClient();
    var r = await client.GetAsync("/orders/1");
    r.EnsureSuccessStatusCode();
  }
}
```

PUBLISH + DOCKER

```
dotnet publish -c Release -o /app \
  -p:PublishReadyToRun=true

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:10.0-noble-
chiseled
WORKDIR /app
COPY --from=build /app .
USER $APP_UID
ENV ASPNETCORE_HTTP_PORTS=8080
ENTRYPOINT ["dotnet", "Shop.Web.dll"]
```

UI COMPONENT LIBRARIES

```
// free / open source (register once in Program.cs)
MudBlazor        AddMudServices()
Radzen.Blazor    AddRadzenComponents()
Blazorise        provider: Bootstrap/Tailwind/...
FluentUI Blazor  AddFluentUIComponents()
Ant Design Blazor / HAVIT / Blazor Bootstrap /
MatBlazor
styling: Bootstrap + Tailwind + Sass + Blazor CSS
isolation
commercial: Telerik / Syncfusion / DevExpress / Ignite
UI
```